Model DBMS Question Paper: Quiz Solutions

1. Canned Transactions

Definition: Canned transactions are standard, pre-specified queries and updates that have been written and tested by application programmers. They are typically used by "naive" or "parametric" end-users (e.g., bank tellers, airline reservation agents) who invoke them using simple commands or GUI buttons.

Responsible User: The **System Analyst** and **Application Programmer** are responsible for developing the specifications and implementing these transactions.

2. Characteristics of the DBMS Approach

The key characteristics distinguishing the DBMS approach from traditional file processing are:

1. **Self-Describing Nature:** Contains a catalog (meta-data) defining the structure and constraints.
2. **Insulation between Programs and Data (Data Independence):** Structure changes don't necessarily break code.
3. **Data Abstraction:** Hides storage details (Physical/Logical levels).
4. **Support for Multiple Views:** Different users see different subsets of data.
5. **Sharing of Data & Multi-user Transaction Processing:** Concurrent access with concurrency control.

3. Set Operations on Relations

Given Relations (Tuples):

- **R1:** `{ (HH, T), (SS, Y), (FF, J), (RR, S), (DD, B) }`
- **R2:** `{ (SS, Y), (RR, S), (JJ, K), (BB, J), (AA, F), (JJ, W), (EE, G) }`

(i) Union ($R1 \cup R2$): All unique tuples from both relations. `{ (HH, T), (SS, Y), (FF, J), (RR, S), (DD, B), (JJ, K), (BB, J), (AA, F), (JJ, W), (EE, G) }`

(ii) Intersection ($R1 \cap R2$): Tuples present in BOTH relations. `{ (SS, Y), (RR, S) }`

(iii) Minus / Set Difference ($R1 - R2$): Tuples in R1 that are NOT in R2. `{ (HH, T), (FF, J), (DD, B) }`

4. ER Notations for Attributes

- **Simple/Atomic Attribute:** Oval.
- **Composite Attribute:** Oval attached to other ovals (e.g., Name attached to First, Last).
- **Multivalued Attribute:** Double Oval.

- **Derived Attribute:** Dashed (Dotted) Oval.
- **Key Attribute:** Oval with the text underlined.

5. Attribute Closure Calculation

Problem: $R(A, B, C, D, E, F, G, H, I)$ **FDs:** $F = \{A \to BC, B \to CD, HB \to I, F \to H, D \to F\}$ **Find** $\{AB\}^+$**:**

1. Start with $\{A, B\}$.
2. Apply $A \to BC$: Add C. Result: $\{A, B, C\}$.
3. Apply $B \to CD$: Add D. Result: $\{A, B, C, D\}$.
4. Apply $D \to F$: Add F. Result: $\{A, B, C, D, F\}$.
5. Apply $F \to H$: Add H. Result: $\{A, B, C, D, F, H\}$.
6. Apply $HB \to I$: We have H and B, so add I. Result: $\{A, B, C, D, F, H, I\}$.

Final Closure $\{AB\}^+ = \{A, B, C, D, F, H, I\}$

6. Join Dependency Constraint

The Join Dependency (JD) specifies the constraint for **Fifth Normal Form (5NF)**. It asserts that a relation can be recreated by joining its projections without losing information or creating spurious tuples. It generalizes the Multivalued Dependency (MVD).

7. Does Elasticsearch Have a Schema?

Answer: Yes, but it is **schema-flexible**. **Justification:** While Elasticsearch allows you to index documents without explicitly defining fields beforehand (Dynamic Mapping), it strictly uses a mapping (schema) internally to define field types (text, keyword, integer). If you don't provide one, it guesses; if you do provide one, it enforces it. It is not purely schema-less.

8. Document-Oriented Database

A Document-Oriented Database is a type of NoSQL database designed to store, retrieve, and manage document-oriented information.

- **Storage Format:** Typically JSON, BSON, or XML.
- **Key Feature:** Data is stored in self-contained documents rather than rows and columns.
- **Example:** MongoDB, CouchDB.

9. Desirable Properties of Transactions

The **ACID** properties:

1. **Atomicity:** All or nothing.
2. **Consistency:** Database remains valid before and after transaction.
3. **Isolation:** Transactions do not interfere with each other.
4. **Durability:** Committed changes are permanent.

10. Serial vs. Nonserial Schedules

- **Serial Schedule:** Transactions are executed one after the other (T1 finishes, then T2 starts). There is no interleaving of operations. It is always consistent but has poor performance.
- **Nonserial Schedule:** Operations from varying transactions are interleaved (T1 reads, T2 reads, T1 writes...). It allows concurrency but may lead to inconsistencies if not serializable.

11. Why Concurrent Execution is Desirable?

1. **Improved Throughput:** More transactions completed per second.
2. **Better Resource Utilization:** CPU and I/O disks can work in parallel (one transaction uses CPU while another waits for Disk I/O).
3. **Reduced Response Time:** Short transactions don't get stuck waiting behind long transactions.

12. Deadlock Detection and Resolution

- **Detection:** The DBMS maintains a **Wait-For Graph**. If the graph contains a **cycle** (e.g., T1 waits for T2, T2 waits for T1), a deadlock exists.
- **Resolution:**
    1. **Victim Selection:** The DBMS selects a transaction to roll back (based on age, cost, or progress) to break the cycle.
    2. **Rollback:** The victim transaction is aborted and restarted later.
    3. **Starvation Prevention:** Ensures the same transaction isn't always picked as the victim.

Part B: Descriptive Questions

2.a. Three-Schema Architecture

Definition: The Three-Schema Architecture (ANSI/SPARC architecture) separates the user applications from the physical database to achieve data independence. It defines three levels:

1. **Internal Level (Physical Schema):** Describes the physical storage structure of the database (e.g., indexes, data compression, block distribution).
2. **Conceptual Level (Logical Schema):** Describes the structure of the whole database for a community of users (e.g., entities, data types, relationships). It hides physical storage details.
3. **External Level (View Schema):** Describes the part of the database that a particular user group is interested in. There can be many external views.

Why Mappings Are Required: Mappings are needed to transform requests and data between these levels:

- **External/Conceptual Mapping:** Maps a user's view to the conceptual schema. It ensures **Logical Data Independence** (changing the logical schema doesn't break external views).
- **Conceptual/Internal Mapping:** Maps the logical schema to physical storage. It ensures **Physical Data Independence** (changing storage structures doesn't require changing the logical schema).

Role of Schema Definition Languages (SDL):

- **Internal SDL:** Used to specify the internal schema (storage details).
- **DDL (Data Definition Language):** Used to specify the conceptual schema.
- **VDL (View Definition Language):** Used to specify user views/mappings.
- *Note:* In modern SQL, DDL handles both conceptual and some internal/external definitions.

2.b. Partial vs. Total Participation

Participation constraints determine whether the existence of an entity depends on its being related to another entity.

Feature	Total Participation (Existence Dependency)	Partial Participation
Definition	Every entity in the entity set **must** be involved in the relationship.	Only **some** entities in the entity set are involved in the relationship.
ER Notation	**Double Line** connecting Entity to Relationship.	**Single Line** connecting Entity to Relationship.
Min Cardinality	Min = 1 (or more).	Min = 0.

Example: Consider relationships between `Employee`, `Department`, and `Project`.

- **Total:** `Employee` **WORKS_FOR** `Department`. (Every employee must belong to a department).
- **Partial:** `Employee` **MANAGES** `Department`. (Not every employee is a manager; only some manage a department).

2.c. Functional Components of DBMS

A DBMS is a complex software system partitioned into modules that deal with specific responsibilities.

Key Components:

1. **Query Processor:**
    - **DDL Compiler:** Processes schema definitions and stores metadata in the System Catalog.

- **DML Compiler / Parser:** Parses queries, checks syntax, and converts them into internal representations.
- **Query Optimizer:** Selects the most efficient execution strategy (plan).
- **Execution Engine:** Executes the plan by making calls to the Storage Manager.

2. **Storage Manager:**
   - **Buffer Manager:** Manages data in main memory (RAM). Decides which data blocks to cache and which to write to disk.
   - **File Manager:** Manages allocation of space on disk structures.
   - **Authorization & Integrity Manager:** Checks user permissions and data constraints.
   - **Transaction Manager:** Ensures ACID properties (Concurrency Control & Recovery).

Diagram of DBMS Architecture:

```
graph TD
    User[Users / Programmers]

    subgraph Query_Processor ["Query Processor"]
        Parser[Parser / Translator]
        Optimizer[Query Optimizer]
        Exec[Execution Engine]
    end

    subgraph Storage_Manager ["Storage Manager"]
        TransMgr[Transaction Manager]
        AuthMgr[Auth & Integrity Manager]
        BufferMgr[Buffer Manager]
        FileMgr[File Manager]
    end

    subgraph Disk_Storage ["Disk Storage"]
        Data[Data Files]
        Indices[Indices]
        Catalog[Data Dictionary / Catalog]
    end

    User --> Parser
    Parser --> Optimizer
    Optimizer --> Exec

    Exec --> BufferMgr
    Exec --> TransMgr
    Exec --> AuthMgr

    TransMgr --> FileMgr
    BufferMgr --> FileMgr

    FileMgr --> Data
    FileMgr --> Indices
    FileMgr --> Catalog
```

3.a. Movie Database ER Diagram

Entities & Attributes:

1. **MOVIE:**
   - **Attributes:** Title, Year (Composite Key {Title, Year}), Length, Plot.
   - **Multivalued Attribute:** Genre.
2. **ACTOR:**
   - **Attributes:** Name, Date_of_Birth (Composite Key {Name, DOB}).
3. **DIRECTOR:**
   - **Attributes:** Name, Date_of_Birth (Composite Key {Name, DOB}).
4. **PRODUCTION_COMPANY:**
   - **Attributes:** Name (Primary Key), Address.
5. **QUOTE:**
   - **Type:** Weak Entity (Dependent on Movie).
   - **Attribute:** Quote_Text.

Relationships:

1. **Produces (1:N):** One Company produces N Movies. One Movie is produced by 1 Company.
2. **Directs (M:N):** One Director directs N Movies. One Movie is directed by M Directors.
3. **Acts_In (M:N):** One Actor appears in N Movies. One Movie has M Actors.
   - **Relationship Attribute:** Role.
4. **Has_Quote (1:N):** One Movie has N Quotes (Weak Relationship).
5. **Spoken_By (N:1):** Each Quote is spoken by exactly one Actor.

ER Diagram:

```
erDiagram
    MOVIE {
        string Title PK
        int Year PK
        int Length
        string Plot
        string[] Genre
    }
    ACTOR {
        string Name PK
        date DOB PK
    }
    DIRECTOR {
        string Name PK
        date DOB PK
    }
    PRODUCTION_COMPANY {
        string Name PK
        string Address
    }
    QUOTE {
        string Text
```

```
}

PRODUCTION_COMPANY ||--|{ MOVIE : "produces"
DIRECTOR }|--|{ MOVIE : "directs"
ACTOR }|--|{ MOVIE : "acts in (Role)"
MOVIE ||--o{ QUOTE : "has"
ACTOR ||--o{ QUOTE : "speaks"
```

3.b. Entity and Referential Integrity Constraints

1. Entity Integrity Constraint:

- **Definition:** No primary key value can be **NULL**.
- **Why Important:** The primary key is used to uniquely identify individual tuples in a relation. If the primary key were NULL, we would not be able to distinguish that tuple from others or reference it reliably. It ensures that every specific entity is identifiable.

2. Referential Integrity Constraint:

- **Definition:** A Foreign Key value in a child table must either match a Primary Key value in the parent table or be NULL.
- **Why Important:** It maintains the consistency of relationships between tables. It prevents "dangling pointers" or invalid references.
  - *Example:* You cannot assign an employee to `Department_ID = 5` if Department 5 does not exist in the Department table. This ensures data quality and logic.

4.a. Relational Algebra Queries

Schema:

- `Hotel(hotelNo, HotelName, city)`
- `Room(roomNo, hotelNo, type, price)`
- `Booking(hotelNo, guestNo, dateFrom, dateTo, roomNo)`
- `Guest(guestNo, guestName, guestAddress)`

(i) List all single rooms with a price below Rs. 2000 per night.

$$\pi_{roomNo,hotelNo,type,price}(\sigma_{type='single' \wedge price<2000}(Room))$$

(ii) List the names and address of all guests.

$$\pi_{guestName,guestAddress}(Guest)$$

(iii) List the price and type of all rooms at the Sagar hotel.

$$\pi_{price,type}(\sigma_{HotelName='Sagar'}(Room \bowtie Hotel))$$

(iv) List all guests currently staying at the Sagar Hotel. *Logic:* We find bookings where the Hotel Name is 'Sagar' AND the current date falls between `dateFrom` and `dateTo`.

$$\pi_{guestName,guestAddress}(Guest \bowtie (\sigma_{HotelName='Sagar' \wedge dateFrom \leq TODAY \wedge dateTo \geq TODAY}(Booki$$

(v) List all the hotels.

$$\pi_{hotelNo,HotelName,city}(Hotel)$$

4.b. DIVISION Operation in Relational Algebra

Concept: The Division operation ($R \div S$) is used for queries involving "universal quantification" (e.g., "for all", "every"). It answers questions like: *"Find entities in R that are associated with **every** entity in S."*

Requirements:

1. **Attributes:** If relation R has attributes (X, Y) and relation S has attributes (Y), then the result of $R \div S$ will have attributes (X).
2. **Subset:** The attributes of the denominator (S) must be a proper subset of the numerator (R).

Representation:

$$R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

Where:

- $\pi_X(R)$: All candidate X values.
- $\pi_X(R) \times S$: All possible combinations of X and Y.
- The subtraction logic finds X values that are *missing* at least one Y value from S, and removes them from the candidate list.

Example: *Query:* Find students who have taken **ALL** required courses.

Relation: Takes (Student, Course)

Student	Course
A	DBMS
A	OS
B	DBMS
C	OS

Relation: Required (Course)

Course
DBMS
OS

Result ($Takes \div Required$):

Student
A

Reason: Only Student 'A' has records for both DBMS and OS.

4.c. Candidate Keys and Super Keys

1. Schema: SALE (salesperson_id, Serial_No, Date, Sale_Price)

- **Assumption:** `Serial_No` uniquely identifies a specific item sold in a specific transaction.
- **Candidate Key:** `{Serial_No}`
- **Super Keys:** Any set containing the candidate key.
  - `{Serial_No}`
  - `{Serial_No, Date}`
  - `{Serial_No, salesperson_id}`
  - `{Serial_No, Date, Sale_Price}`

2. Schema: SALESPERSON (Salesperson_id, Name, Phone)

- **Assumption:** `Salesperson_id` is the unique identifier for an employee.
- **Candidate Key:** `{Salesperson_id}`
- **Super Keys:** Any set containing the candidate key.
  - `{Salesperson_id}`
  - `{Salesperson_id, Name}`
  - `{Salesperson_id, Name, Phone}`

5.a. SQL Queries (Suppliers-Parts-Catalog)

Schema:

- `Suppliers(sid: int, Sname: string, address: string)`
- `Parts(Pid: int, Pname: string, color: string)`
- `Catalog(sid: int, pid: int, cost: real)`

(i) Find the names of the suppliers who supply some red part.

```
SELECT DISTINCT S.sname
FROM Suppliers S
JOIN Catalog C ON S.sid = C.sid
JOIN Parts P ON C.pid = P.pid
WHERE P.color = 'red';
```

(ii) Find the SIDs of suppliers who supply some red or green part.

```
SELECT DISTINCT C.sid
FROM Catalog C
JOIN Parts P ON C.pid = P.pid
WHERE P.color IN ('red', 'green');
```

(iii) Find the SIDs of suppliers who supply some red part or are at '221 Parker Street'.
Logic: We use `LEFT JOIN` to ensure we include suppliers who have the correct address but might not supply any parts (or any red parts).

```
SELECT DISTINCT S.sid
FROM Suppliers S
LEFT JOIN Catalog C ON S.sid = C.sid
LEFT JOIN Parts P ON C.pid = P.pid
WHERE P.color = 'red'
   OR S.address = '221 Parker Street';
```

5.b. Normalization (Key, 2NF, 3NF)

Relation: $R = (A, B, C, D, E, F, G, H, I, J, K, L)$ **FDs:**
$F = \{AB \to C, B \to F, F \to G, H, D \to E, I, J, B \to K, L, K \to L\}$

(a) Find a Key for R

We calculate the closure of $\{A, B, D\}$:

1. **Start:** $\{A, B, D\}$
2. Use $AB \to C$: $\{A, B, C, D\}$
3. Use $B \to F$: $\{A, B, C, D, F\}$
4. Use $F \to G, H$: $\{A, B, C, D, F, G, H\}$
5. Use $D \to E, I, J$: $\{A, B, C, D, E, F, G, H, I, J\}$
6. Use $B \to K, L$: $\{A, B, C, D, E, F, G, H, I, J, K, L\}$

Result: $\{A, B, D\}$ is the **Candidate Key**.

(b) 2NF Decomposition

Rule: A relation is in 2NF if it is in 1NF and contains **no partial dependencies** (non-prime attributes must not depend on a proper subset of the key).

- **Key:** $\{A, B, D\}$
- **Partial Dependencies found:**
  - $AB \to C$ (Depends on subset AB)
  - $B \to F, K, L$ (Depends on subset B)
  - $D \to E, I, J$ (Depends on subset D)

Decomposition: We create separate relations for these partial dependencies:

1. **R1(A, B, C)** (from $AB \to C$)
2. **R2(B, F, K, L)** (from $B \to F, K, L$)
3. **R3(D, E, I, J)** (from $D \to E, I, J$)
4. **R4(A, B, D)** (Residual relation with Key)

Note on Transitive Dependencies: FDs like $F \to G, H$ (where F depends on B) create transitive chains. In 2NF, these usually stay with the determinant (F is in R2, so G, H conceptually follow, though strictly they can be split now or in 3NF).

(c) 3NF Decomposition

Rule: A relation is in 3NF if it is in 2NF and contains **no transitive dependencies** (non-prime attributes must not depend on other non-prime attributes).

Transitive Dependencies to Resolve:

- In R2, we have $B \to F$ and $F \to G, H$. This means G, H transitively depend on B.
  - **Action:** Create **R3(F, G, H)**.
- $D \to E, I, J$ is already clean (Direct dependency on key D).
- $AB \to C$ is already clean.

Final 3NF Decomposition:

1. **R1(A, B, C)**
2. **R2(B, F, K, L)** (Keeps $B \to F$ and $B \to K, L$)
3. **R3(F, G, H)** (Resolves $F \to G, H$)
4. **R4(D, E, I, J)**
5. **R5(A, B, D)** (Key Relation to ensure lossless join)

6.a. Project-Join Normal Form (PJNF / 5NF)

Definition: Project-Join Normal Form (also known as **Fifth Normal Form** or **5NF**) is a level of database normalization designed to reduce redundancy in relational databases recording multi-way relationships. A relation R is in **5NF** if:

1. It is already in **4NF** (Fourth Normal Form).

2. Every **Join Dependency (JD)** that holds for R is implied by the candidate keys of R.

Concept: It deals with cases where a relation can be reconstructed (losslessly joined) from three or more smaller projections, but not by just two (which would be covered by 4NF/MVDs). This usually occurs in cyclic relationships.

Example: Consider a relation `Supply(Supplier, Part, Project)`.

- **Rule:** If a Supplier S supplies a Part P, and Part P is used in Project J, and Supplier S works with Project J, then we record that S supplies P to J.
- **Problem:** This creates a Join Dependency $*(\{S, P\}, \{P, J\}, \{S, J\})$. Storing them in one table leads to redundancy.
- **Normalization:** We decompose `Supply` into three relations:
  1. `R1(Supplier, Part)`
  2. `R2(Part, Project)`
  3. `R3(Supplier, Project)`
- This decomposition is in 5NF because the information is fully represented by joining these three tables, and no single table contains redundancy.

6.b. Trivial vs. Non-Trivial Multi-Valued Dependency (MVD)

A Multi-Valued Dependency (MVD), denoted $X \twoheadrightarrow Y$, signifies that for a given value of X, there is a specific set of Y values that are independent of the other attributes in the relation.

Feature	Trivial MVD	Non-Trivial MVD
Definition	An MVD $X \twoheadrightarrow Y$ is trivial if either:	An MVD $X \twoheadrightarrow Y$ is non-trivial if:
	1. Y is a subset of X ($Y \subseteq X$).	1. Y is NOT a subset of X.
	2. X and Y together form the entire relation ($X \cup Y = R$).	2. $X \cup Y$ is NOT the entire relation (there is at least one other attribute Z).
Redundancy	Does not imply redundancy or normalization issues.	Usually implies redundancy and requires decomposition (4NF).

Example: Consider Relation `Course_Resources(Course, Teacher, Textbook)`.

- **Trivial:** `Course, Teacher` $\twoheadrightarrow$ `Teacher`. (Subset condition met).
- **Non-Trivial:** `Course` $\twoheadrightarrow$ `Teacher`.
  - If a Course has a set of Teachers and a set of Textbooks, and they are independent, this is a non-trivial MVD because `Teacher` is not a subset of `Course`, and there is another attribute (`Textbook`) left over.

6.c. SQL Nested Query Operators

These operators are used in the `WHERE` clause to compare values against the result of a subquery.

Schema for Examples:

- `Employee(EmpID, Name, Salary, DeptID)`
- `Department(DeptID, DeptName)`

1. IN

- **Usage:** Checks if a value matches any value in a list or subquery result.
- **Example:** Find employees who work in the 'IT' or 'HR' departments.

```
SELECT Name FROM Employee
WHERE DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('IT', 'HR')
```

2. EXISTS

- **Usage:** Returns `TRUE` if the subquery returns at least one row. Used for correlation.
- **Example:** Find departments that have at least one employee.

```
SELECT DeptName FROM Department D
WHERE EXISTS (SELECT * FROM Employee E WHERE E.DeptID = D.DeptID);
```

3. UNIQUE

- **Usage:** Returns `TRUE` if the subquery result contains no duplicate tuples.
- **Example:** Find departments that have exactly one employee.

```
SELECT DeptName FROM Department D
WHERE UNIQUE (SELECT * FROM Employee E WHERE E.DeptID = D.DeptID);
```

4. ANY (or SOME)

- **Usage:** Returns `TRUE` if the comparison holds for **at least one** value returned by the subquery.
- **Example:** Find employees who earn more than *any* employee in Dept 5.

```
SELECT Name FROM Employee
WHERE Salary > ANY (SELECT Salary FROM Employee WHERE DeptID = 5);
```

5. ALL

- **Usage:** Returns `TRUE` if the comparison holds for **every** value returned by the subquery.
- **Example:** Find employees who earn more than *all* employees in Dept 5.

```
SELECT Name FROM Employee
WHERE Salary > ALL (SELECT Salary FROM Employee WHERE DeptID = 5);
```

7.a. Concurrency Conflicts and Strict 2PL

Transactions:

- $T1 : R(X), R(Y), W(X)$
- $T2 : R(X), R(Y), W(X), W(Y)$

(i) Write-Read (WR) Conflict (Dirty Read)

- **Definition:** T1 writes an object, T2 reads it before T1 commits.
- **Example Schedule:**

$$r_1(X), r_1(Y), w_1(X), \mathbf{r_2(X)}, r_2(Y) \dots$$

- **Strict 2PL Prevention:**
  - When $T1$ executes $w_1(X)$, it must obtain an **Exclusive Lock (X-lock)** on X.
  - Under Strict 2PL, this lock is held until $T1$ commits.
  - When $T2$ attempts $r_2(X)$, it requests a **Shared Lock (S-lock)** on X.
  - The S-lock is denied because an X-lock is held by $T1$. $T2$ is forced to wait, preventing the schedule.

(ii) Read-Write (RW) Conflict (Unrepeatable Read)

- **Definition:** T1 reads an object, T2 writes it before T1 finishes (causing T1 to potentially read a different value later).
- **Example Schedule:**

$$r_1(X), \mathbf{w_2(X)}, w_2(Y), r_1(Y) \dots$$

- **Strict 2PL Prevention:**
  - When $T1$ executes $r_1(X)$, it obtains a **Shared Lock (S-lock)** on X.
  - Under Strict 2PL, this lock is held until $T1$ commits.
  - When $T2$ attempts $w_2(X)$, it requests an **Exclusive Lock (X-lock)**.
  - The X-lock is denied because an S-lock is held by $T1$. $T2$ blocks, preventing the schedule.

(iii) Write-Write (WW) Conflict (Lost Update)

- **Definition:** T1 writes an object, T2 writes it before T1 commits (overwriting T1's value).

- **Example Schedule:**

$$w_1(X), \mathbf{w_2(X)}, w_2(Y) \dots$$

- **Strict 2PL Prevention:**
  - When $T1$ executes $w_1(X)$, it obtains an **Exclusive Lock (X-lock)** on X.
  - Under Strict 2PL, this lock is held until $T1$ commits.
  - When $T2$ attempts $w_2(X)$, it requests an **X-lock**.
  - The X-lock is denied because an X-lock is already held by $T1$. $T2$ blocks.

7.b. Two-Phase Locking (2PL) Techniques

The Two-Phase Locking protocol ensures serializability by defining how transactions acquire and release locks.

The Two Phases:

1. **Growing Phase (Expanding):**
   - The transaction acquires all the locks (Shared or Exclusive) it needs.
   - **Constraint:** The transaction cannot release any lock during this phase.
2. **Shrinking Phase (Contracting):**
   - The transaction releases its locks.
   - **Constraint:** Once the first lock is released, the transaction cannot acquire any new locks.

Demonstration Example: Consider $T1$ transferring $50 from Account A to B.

Step	Transaction Operation	Locking Action (Growing)	Locking Action (Shrinking)
1	`Read(A)`	`Lock_X(A)` (Acquired)	
2	`A = A - 50`		
3	`Write(A)`		
4	`Read(B)`	`Lock_X(B)` (Acquired)	
5	**Locked Point**	**(Max locks held)**	
6	`B = B + 50`		`Unlock(A)` (Released)
7	`Write(B)`		
8	**Commit**		`Unlock(B)` (Released)

Variations:

- **Basic 2PL:** Allows releasing locks (step 6) before commit, but risks cascading aborts.
- **Strict 2PL:** All Exclusive locks must be held until **Commit/Abort**. (Prevents dirty reads and cascading aborts).
- **Rigorous 2PL:** All locks (Shared and Exclusive) must be held until **Commit/Abort**.

8.a. Transaction States and State Diagram

Transaction States: A transaction must be in one of the following states to ensure the properties of atomicity and consistency:

1. **Active:** The initial state; the transaction stays in this state while it is executing read or write operations.
2. **Partially Committed:** The transaction enters this state after the final statement has been executed. It is waiting to ensure all records can be safely written to disk.
3. **Failed:** The state entered after the discovery that normal execution can no longer proceed (due to hardware failure or logical error).
4. **Aborted:** The state after the transaction has been rolled back and the database restored to its state prior to the start of the transaction.
5. **Committed:** The state after successful completion. The changes are now permanently recorded in the database.
6. **Terminated:** The final state where the transaction leaves the system.

State Diagram:

```
stateDiagram-v2
    [*] --> Active : Begin
    Active --> PartiallyCommitted : End Transaction
    Active --> Failed : Error
    PartiallyCommitted --> Committed : Commit
    PartiallyCommitted --> Failed : Error
    Failed --> Aborted : Rollback
    Aborted --> Terminated
    Committed --> Terminated
    Terminated --> [*]
```

8.b. Need for Concurrency Control

Why it is needed: Concurrency control is the management of concurrent execution of transactions. It is needed to ensure:

1. **Isolation:** Transactions do not interfere with one another.
2. **Consistency:** The database remains in a consistent state despite interleaved operations. Without it, several data processing anomalies can occur, such as the **Lost Update**, **Dirty Read**, and **Unrepeatable Read**.

Demonstration (The Lost Update Problem): Consider two transactions, $T1$ and $T2$, both trying to withdraw money from the same account (X) with an initial balance of 1000.

Time	Transaction T1	Transaction T2	Value of X in DB
1	`Read(X)` (Reads 1000)		1000
2		`Read(X)` (Reads 1000)	1000
3	`X = X - 100` (Local: 900)		1000
4		`X = X - 50` (Local: 950)	1000
5	`Write(X)` (Writes 900)		**900**
6		`Write(X)` (Writes 950)	**950**

Result:

- $T1$ updates the balance to 900.
- $T2$ overwrites it with 950 (because it calculated based on the stale read of 1000).
- The update by $T1$ is **lost**. The final balance should be 850, but it is 950. Concurrency control (locking) would prevent this by forcing $T2$ to wait.

8.c. Serializability Check (Precedence Graph)

Schedule S1:

$$R_1(X), R_2(Z), R_1(Z), R_3(X), R_3(Y), W_1(X), W_3(Y), R_2(Y), W_2(Z), W_2(Y)$$

Algorithm for Conflict Serializability:

1. **Create a node** for each transaction ($T1, T2, T3$).
2. **Draw directed edges** ($Ti \rightarrow Tj$) if there is a conflict. A conflict exists if:
   - Operations belong to different transactions ($Ti \neq Tj$).
   - They access the same data item.
   - At least one operation is a **Write**.
   - Op_i occurs before Op_j in the schedule.
3. **Check for Cycles:** If the graph has a cycle, it is **not** serializable. If acyclic, it is serializable.

Analysis of Conflicts in S1:

1. **Item X:**
   - $R_3(X)$ (step 4) occurs before $W_1(X)$ (step 6).
   - **Conflict:** $T3 \rightarrow T1$ (Read-Write)

2. **Item Z:**
    - $R_1(Z)$ (step 3) occurs before $W_2(Z)$ (step 9).
    - **Conflict:** $T1 \rightarrow T2$ (Read-Write)
    - (*Note:* $R_2(Z)$ is in T2, so no conflict with $W_2(Z)$).
3. **Item Y:**
    - $R_3(Y)$ (step 5) occurs before $W_2(Y)$ (step 10).
    - **Conflict:** $T3 \rightarrow T2$ (Read-Write)
    - $W_3(Y)$ (step 7) occurs before $R_2(Y)$ (step 8).
    - **Conflict:** $T3 \rightarrow T2$ (Write-Read)
    - $W_3(Y)$ (step 7) occurs before $W_2(Y)$ (step 10).
    - **Conflict:** $T3 \rightarrow T2$ (Write-Write)

Precedence Graph:

```
graph LR
    T3((T3)) --> T1((T1))
    T1 --> T2((T2))
    T3 --> T2
```

Conclusion:

- Edges: $T3 \rightarrow T1$, $T1 \rightarrow T2$, $T3 \rightarrow T2$.
- Path: $T3 \rightarrow T1 \rightarrow T2$.
- **Cycle Check:** There are **NO cycles** in the graph.
- **Result:** The schedule **IS SERIALIZABLE**.
- **Equivalent Serial Order:** $T3 \rightarrow T1 \rightarrow T2$.

9.a. Tokenizer and Index in Elasticsearch

1. Tokenizer: A **Tokenizer** is a core component of text analysis in Elasticsearch. It receives a stream of characters, breaks it up into individual terms (tokens), and outputs a stream of tokens.

- **Function:** It is responsible for splitting raw text into words or terms based on specific rules (e.g., whitespace, punctuation).
- **Usage:** Used during both **Indexing** (when saving data) and **Searching** (when querying data).
- **Examples:** `Standard Tokenizer` (splits on word boundaries), `Whitespace Tokenizer` (splits on spaces).

2. Index: An **Index** in Elasticsearch is a logical namespace (container) for storing related documents.

- **Analogy:** It is roughly equivalent to a **Database** in a relational system.

- **Structure:** Internally, an index is split into **shards**, which contain inverted indices.
- **Relationship:** When a document is added to an Index, the text fields are processed by the Tokenizer to create the Inverted Index, enabling fast full-text search.

9.b. Key Features of MongoDB

MongoDB is a popular NoSQL database known for its flexibility and scalability. Its key features include:

1. **Document-Oriented Storage:** Stores data in **BSON** (Binary JSON) format. Data is stored in documents rather than rows, aligning well with object-oriented code.
2. **Schema Flexibility:** It has a dynamic schema. Different documents in the same collection can have different fields, and fields can be added on the fly.
3. **High Scalability (Sharding):** Supports horizontal scaling by distributing data across multiple servers (sharding) to handle large datasets and high throughput.
4. **High Availability (Replica Sets):** Uses replication to provide automatic failover and data redundancy. If a primary node fails, a secondary is automatically elected.
5. **Rich Query Language:** Supports complex ad-hoc queries, indexing (on any field), and a powerful aggregation framework for data processing.

9.c. Hadoop Distributed Architecture (HDFS)

The Hadoop Distributed File System (HDFS) is the storage layer of Hadoop, designed to run on commodity hardware. It follows a **Master-Slave Architecture**.

Core Components:

1. **NameNode (Master):**
   - The centerpiece of HDFS. It manages the **File System Namespace** (metadata) and regulates access to files by clients.
   - It stores the mapping of blocks to DataNodes in memory (RAM) for fast access.
   - It does **not** store actual data.
2. **DataNode (Slave):**
   - The worker nodes. They store the actual **Data Blocks**.
   - They are responsible for serving read and write requests from clients.
   - They perform block creation, deletion, and replication upon instruction from the NameNode.
   - They send periodic **Heartbeats** to the NameNode to report their health.
3. **Secondary NameNode:**
   - **Not a backup** (despite the name).
   - It periodically merges the namespace image with the edit log to prevent the edit log from getting too large (Checkpointing).

4. **Blocks & Replication:**
    - Files are split into large blocks (default 128MB).
    - Each block is replicated (default 3 copies) across different DataNodes to ensure fault tolerance.

Architecture Diagram:

```
graph TD
    Client[Client Application]

    subgraph Master_Node
    NN[NameNode<br/>(Metadata & Block Map)]
    end

    subgraph Slave_Nodes
    DN1[DataNode 1<br/>Block A, B]
    DN2[DataNode 2<br/>Block A, C]
    DN3[DataNode 3<br/>Block B, C]
    end

    %% Interactions
    Client -- 1. Metadata Ops --> NN
    Client -- 2. Read/Write Blocks --> DN1
    Client -- 2. Read/Write Blocks --> DN2

    DN1 -- Heartbeat --> NN
    DN2 -- Heartbeat --> NN
    DN3 -- Heartbeat --> NN

    style NN fill:#f9f,stroke:#333
    style DN1 fill:#bbf,stroke:#333
    style DN2 fill:#bbf,stroke:#333
```

10.a. Types of Data

Big Data and modern database systems handle three main categories of data based on their structure and organization.

i) Structured Data

Definition: Data that is organized in a highly specific, fixed format (rigid schema). It is typically stored in tabular form with rows and columns.

- **Characteristics:**
    - Predefined schema (data types, lengths).
    - Easy to search and query using SQL.
    - Highly organized.
- **Storage:** Relational Databases (RDBMS) like MySQL, Oracle, PostgreSQL.

- **Example:** An `Employee` table. | EmpID | Name | Dept | Salary | | :--- | :--- | :--- | :--- | | 101 | John | IT | 50000 | | 102 | Jane | HR | 55000 |

ii) Semi-Structured Data

Definition: Data that does not reside in a rigid table structure but has some organizational properties like tags, markers, or key-value pairs to separate elements and enforce hierarchies.

- **Characteristics:**
  - Flexible or dynamic schema (Schema-on-read).
  - "Self-describing" (contains metadata within the data).
  - Hierarchical or nested structure.
- **Storage:** NoSQL Databases (MongoDB, Cassandra), XML files.
- **Example:** A JSON document.

```
{
  "name": "John Doe",
  "contact": {
    "email": "john@example.com",
    "phone": ["123-456", "987-654"]
  },
  "active": true
}
```

iii) Unstructured Data

Definition: Data that has no predefined format or organization. It does not fit neatly into databases with rows and columns.

- **Characteristics:**
  - No identifiable internal structure.
  - Difficult to search and analyze using traditional tools.
  - Makes up the majority (approx. 80%) of data generated today.
- **Storage:** Data Lakes, Blob Storage, Hadoop HDFS.
- **Examples:**
  - Multimedia (Images, Videos, Audio).
  - Text files (PDFs, Word docs, Email bodies).
  - Social media posts, sensor logs.

10.b. MongoDB CRUD Operations

Context: MongoDB is a document-oriented NoSQL database. Operations are performed on **Collections** (tables) containing **Documents** (rows). **Example Application:** A Student Management System using a collection named `students`.

1. Create (Insert)

Used to add new documents to a collection.

- `insertOne()` : Inserts a single document.
- `insertMany()` : Inserts multiple documents.

Example:

```
// Add a single student
db.students.insertOne({
    "name": "Ravi Kumar",
    "age": 21,
    "course": "CSE",
    "marks": 85
});
```

2. Read (Find)

Used to query or fetch documents from a collection.

- `find()` : Returns all documents matching the query criteria.
- `findOne()` : Returns the first document matching the criteria.

Example:

```
// Find all students in 'CSE' course
db.students.find({ "course": "CSE" });

// Find students with marks greater than 80 (Filtering)
db.students.find({ "marks": { $gt: 80 } });
```

3. Update (Modify)

Used to modify existing documents.

- `updateOne()` : Updates the first matching document.
- `updateMany()` : Updates all matching documents.
- **Operator:** Uses `$set` to modify fields without overwriting the whole document.

Example:

```
// Update Ravi's marks to 90
db.students.updateOne(
    { "name": "Ravi Kumar" },   // Filter
    { $set: { "marks": 90 } }   // Update Action
);
```

4. Delete (Remove)

Used to remove documents from a collection.

- `deleteOne()` : Removes the first matching document.
- `deleteMany()` : Removes all matching documents.